

FINAL PROJECT REPORT

By:

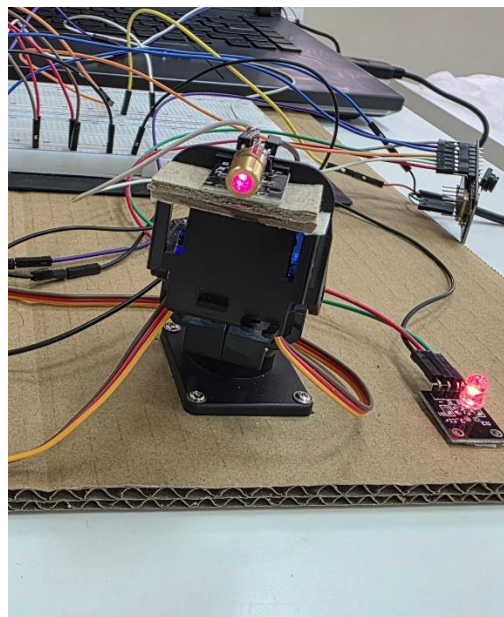
Mario Aouad (231500)

Kevin (El) Khoury (232283)

Andre (El) Cham (233045)

Supervisors:

Dr. Ahmad Audi



DATE

14/5/2026

Ecole Supérieure d'Ingénieurs de Beyrouth

Université Saint-Joseph de Beyrouth

Table of Contents

1. Introduction	3
2. Dataset	4
3. Hardware Architecture and Embedded Components	5
4. Methodology	7
5. Experiments and Results	11
6. Discussion	15
7. Conclusion.....	16
8. References	17

1.Introduction

Computer vision has become an important field in modern automation because it allows machines to interpret visual information from cameras and react to real-world environments. Applications such as object detection, pose estimation, surveillance, robotics, and autonomous systems all rely on the ability of a model to detect and localize objects accurately in images or video streams. In this project, we focus on building a computer vision system capable of detecting humans in real time and using the detected position to support an automated tracking mechanism.

The problem addressed in this project is the detection and localization of humans from camera input for a sentry-style tracking system. Instead of manually controlling the direction of the system, the goal is to allow the camera and vision model to identify a person in the frame and estimate their position. This information can then be used by the control system to decide how the hardware should move. The project therefore combines computer vision, software processing, and hardware communication into one complete pipeline.

This problem is meaningful because real-time human detection is widely used in security systems, smart cameras, robotics, and human-computer interaction. A system that can correctly identify a human target and estimate their position can be extended to many practical applications, such as automated monitoring, assistive robotics, or interactive embedded systems. However, this task also presents several challenges. The model must detect humans under different lighting conditions, distances, body poses, and backgrounds. In addition, the system should operate fast enough to be useful in real time, especially if it is connected to physical hardware.

Several computer vision approaches can be used for this type of task. Classical computer vision methods, such as Histogram of Oriented Gradients combined with Support Vector Machines, were commonly used for pedestrian detection in the past. However, modern deep learning methods usually achieve stronger performance because they learn visual features directly from data. Object detection models such as YOLO, SSD, and Faster R-CNN are widely used to detect objects and humans in images. YOLO-based models are especially useful for real-time applications because they are designed to provide fast predictions while maintaining good detection accuracy.

In this project, we use a deep learning-based detection approach to identify humans in image frames. The system processes video input detects the human target and extracts the target's position in the image. This position can then be translated into commands for the hardware component. The project is evaluated quantitatively using appropriate detection

metrics such as precision, recall, F1-score, and mean average precision, while also including visual examples of successful detections and failure cases.

The main objective of this report is to explain the full development of the system, including the dataset used, preprocessing steps, model choice, implementation pipeline, experiments, results, and limitations.

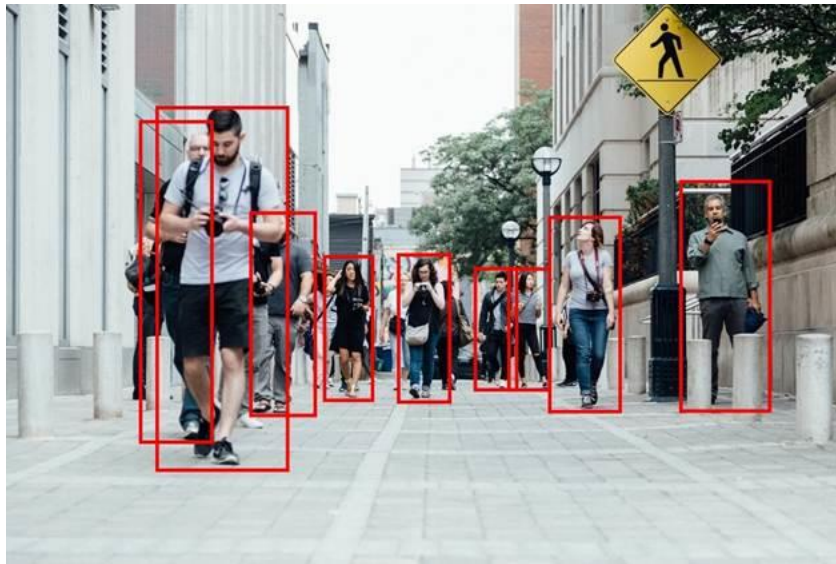


Figure 1 Computer vision system detecting humans

2. Dataset

The project did not require training a new model from scratch. Instead, pretrained pose estimation models were evaluated using both public benchmark data and custom data collected from the project's own camera setup. The dataset was therefore used for model selection, performance validation, and tracking evaluation rather than training.

COCO 2017 validation dataset was used to evaluate theoretical pose estimation accuracy. COCO is a standard public dataset for object detection and human pose estimation, containing people in different poses, scales, lighting conditions, and occlusion levels. The relevant annotations were the human keypoint labels, which define 17 body points such as the nose, eyes, ears, shoulders, elbows, wrists, hips, knees, and ankles.

This COCO benchmark allowed comparison between YOLO-Pose variants and MediaPipe using metrics such as mean Average Precision, F1-score, Object Keypoint Similarity, and Percentage of Correct Keypoints. This was important because the turret depends on stable

and accurate body keypoints to decide where to aim. If the detected keypoints are unstable, the servo movement and laser targeting also become unreliable.

In addition to COCO, a custom live-camera dataset was collected using the same camera setup intended for the final system. It contained 1,055 sequential webcam frames of a human subject moving in front of the camera under realistic conditions such as motion blur, partial occlusion, webcam noise, lighting variation, and body movement.

This custom dataset was used to evaluate real-time inference performance, including FPS, inference latency, confidence score, and confidence stability. Since the turret needed to react smoothly to a moving target, the vision pipeline had to run above 30 FPS and remain reliable under actual webcam conditions.

Also this custom made dataset was used to compare ByteTrack and DeepSORT. We evaluated according to tracking accuracy, identity consistency, false positives, missed detections, and bounding box precision.

ByteTrack was selected because it provided better tracking stability and fewer false positives. This was important for a physical turret because incorrect detections could cause sudden unwanted servo movement or make the system aim at the wrong target.

Preprocessing varied depending on the dataset. COCO images and keypoint annotations were loaded and compared against model predictions, custom webcam frames were extracted and passed through the final vision pipeline, and tracking frames were labeled with bounding boxes and converted into the required tracking evaluation format.

3. Hardware Architecture and Embedded Components

The hardware architecture was designed to separate heavy computation from physical actuation. The laptop or PC acts as the central compute node, while the ESP32 acts as the low-level motor controller. This separation improves latency and reliability because the computer handles the expensive vision and kinematics calculations, while the ESP32 only receives final angle commands and applies them to the servos.

The central compute node is the main processing unit of the system. It uses the local camera feed as the visual input and runs the full Python vision pipeline. This includes human detection, pose estimation, target point selection, tracking, filtering, depth estimation, spatial calibration, and inverse kinematics. A dedicated GPU, such as an RTX 4060, allows the deep learning model to run in real time. After calculating the required pan and tilt angles, the compute node sends the commands to the ESP32 through a high-speed serial connection.

The ESP32-CAM is used as the embedded actuation controller. Although the board includes an OV2640 camera sensor, in this project it is repurposed mainly as a motor control node rather than as the primary vision processor. The ESP32 receives serial text commands from the Python backend and converts them into PWM signals for the servo motors. This avoids depending on Wi-Fi streaming or wireless control, reducing communication delay and making the system more responsive.

The actuation system uses two SG90 micro servos arranged in a two-degree-of-freedom pan-tilt mechanism. One servo controls horizontal yaw movement, while the other controls vertical pitch movement. These servos receive angle commands calculated by the inverse kinematics module and physically orient the laser toward the selected target point.

The targeting payload is a 5V laser diode mounted on the pan-tilt assembly. Its purpose is to provide a visible indication of where the turret is aiming. The laser is mounted as parallel as possible to the camera direction to reduce alignment error. Since even small offsets can affect accuracy, the system also includes trim and calibration parameters to compensate for physical misalignment.

The mechanical chassis consists of an SG90 pan-tilt camera bracket or an equivalent rigid mount. This structure holds the servos, camera, and laser in a fixed geometric relationship. A stable mechanical frame is important because the software calibration assumes that the camera, laser, and turret rotation center maintain the same relative positions during operation.

A 5V LEDs is used as status indicators. The LEDs is connected to ESP32 GPIO pin and show the current system state. The Green color LED represent scanning, while the Red indicate that a target is locked.

The power system uses a regulated 5V supply routed through a breadboard. This is important because servo motors can draw more current than the ESP32 logic pins can safely provide. By powering the servos from a dedicated 5V source while maintaining a common ground with the ESP32, the system reduces the risk of voltage drops, brownouts, and unstable servo behavior.

The data tether is a USB-to-TTL serial cable connecting the compute node to the ESP32. It acts as the real-time communication bridge of the system. The Python backend sends commands in the format pan,tilt,state, and the ESP32 firmware parses these values to update the servos and payload. This wired connection was chosen because it is more predictable and lower latency than wireless communication.

DuPont jumper wires are used for logic routing between the ESP32, servos, LEDs, laser module, and power distribution board. These wires provide the necessary GPIO connections and common ground links. Maintaining a shared ground between the compute interface, ESP32, servo power, and payload is essential for stable signal interpretation and reliable hardware behavior.

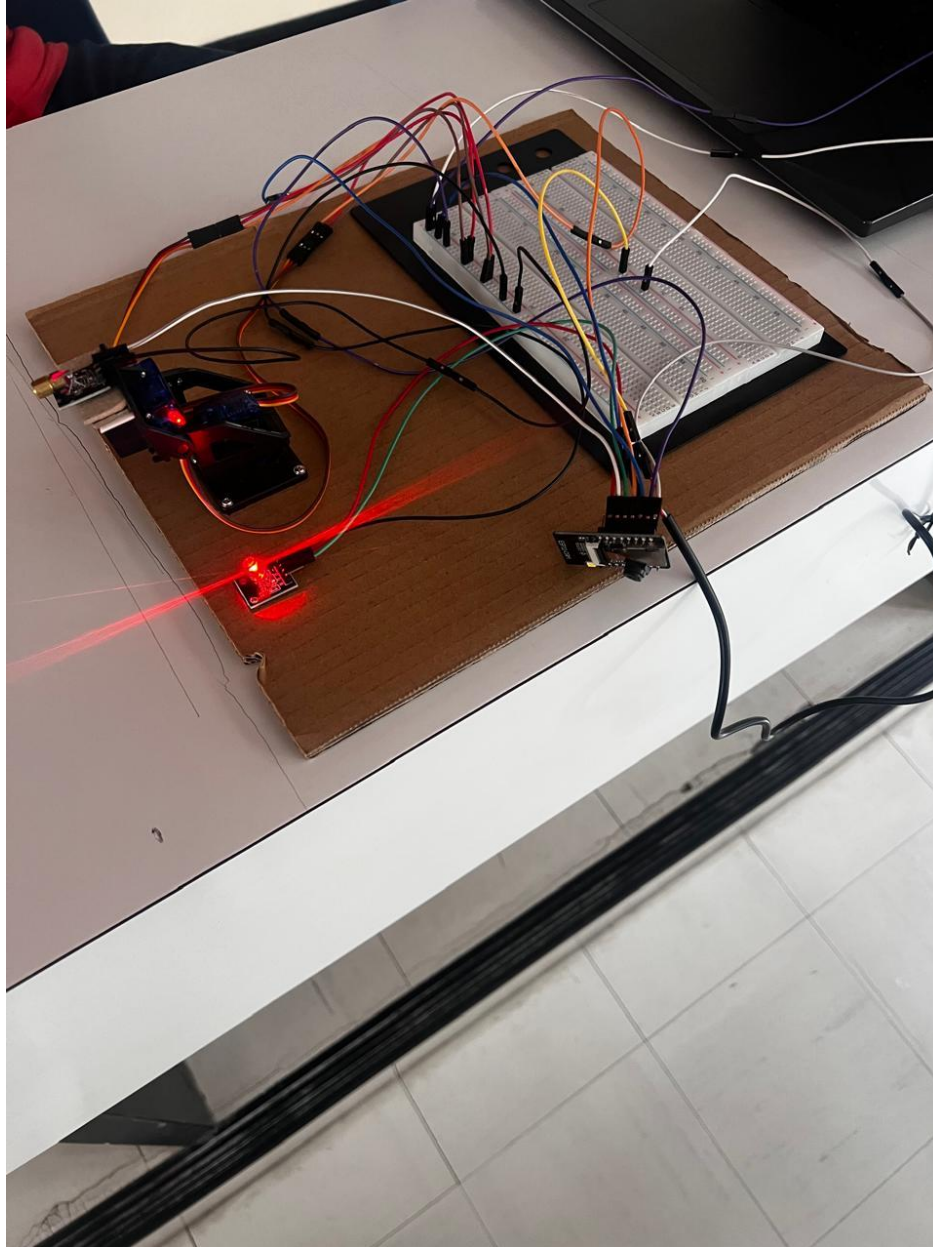


Figure 2 Hardware Integration of ESP32, Breadboard, Laser Module, and Pan-Tilt Servos

4. Methodology

The system was designed as three connected modules: a Python vision backend, an ESP32 actuation firmware, and a React frontend dashboard. The Python backend is responsible for the main intelligence of the system, including camera acquisition, pose estimation, tracking, filtering, depth estimation, spatial calibration, inverse kinematics, API communication, and serial transmission. The ESP32 acts as the low-level hardware controller, receiving

calculated servo angles from the Python backend and applying them to the pan-tilt mechanism. The frontend provides a monitoring and control interface, allowing the user to view the live feed, system state, FPS, confidence, servo positions, and selected target mode.

The vision pipeline begins by capturing frames from the camera using OpenCV. Each frame is processed by YOLO11m-Pose, which detects the human subject and extracts body keypoints. The system can target either the head or the chest depending on the selected mode. For head targeting, the nose keypoint is used as the target point. For chest targeting, the midpoint between the left and right shoulder keypoints is used. This gives the system a clear 2D pixel coordinate that represents the point the turret should aim toward.

To improve stability across frames, ByteTrack is used as the tracking algorithm. Pose estimation models suffer from frame amnesia, meaning they process each frame independently and do not naturally remember that the person detected in the current frame is the same person from the previous frame. This can cause unstable behavior when the subject moves, becomes partially occluded, or appears blurry for a short period. Without tracking, the system could react too strongly to frame-by-frame detection changes, leading to unstable servo movement or unwanted target switching. ByteTrack solves this by maintaining a persistent target identity across consecutive frames, making the target selection more stable and reliable.

After the target point is selected, the raw pixel coordinates are filtered before being used for movement. A median filter reduces sudden outlier detections, while an exponential moving average smooths the target motion over time. Depth is then estimated using the apparent shoulder width in the image and an assumed real-world shoulder width. This provides an approximate distance from the camera to the person, which is necessary because the system uses a normal 2D camera rather than a depth sensor.

The filtered 2D target point and estimated depth are then passed into the spatial calibration stage. Using the camera's field of view and focal length, the system converts the target pixel into a 3D position relative to the camera. Since the laser and turret rotation center are not exactly at the camera lens, the measured physical offset between the camera and the turret is applied. This converts the point from camera-space into turret-space, giving the system a more realistic target coordinate for aiming.

The inverse kinematics module converts the turret-space coordinate into pan and tilt servo angles. Servo direction correction is applied to account for how the motors are physically mounted, and trim values are used to compensate for small mechanical alignment errors.

The final angles are clamped within safe limits to prevent the servos from moving beyond the usable range of the pan-tilt bracket.

Once the angles are calculated, the Python backend sends them to the ESP32 through a serial connection using the format pan,tilt,state. The ESP32 parses the command, updates the servo positions, and controls the laser and LEDs according to the current system state. This separation keeps the ESP32 firmware lightweight while allowing the more expensive vision and math operations to run on the computer.

The frontend dashboard communicates with the backend through WebSocket telemetry and an MJPEG video stream. It displays the live camera feed, system status, confidence, FPS, and servo angles. A green virtual laser dot is drawn directly on top of the live feed to show the predicted point where the laser should hit. This makes debugging and calibration easier because the user can compare the computed laser position with the real scene instead of viewing it on a separate black canvas.

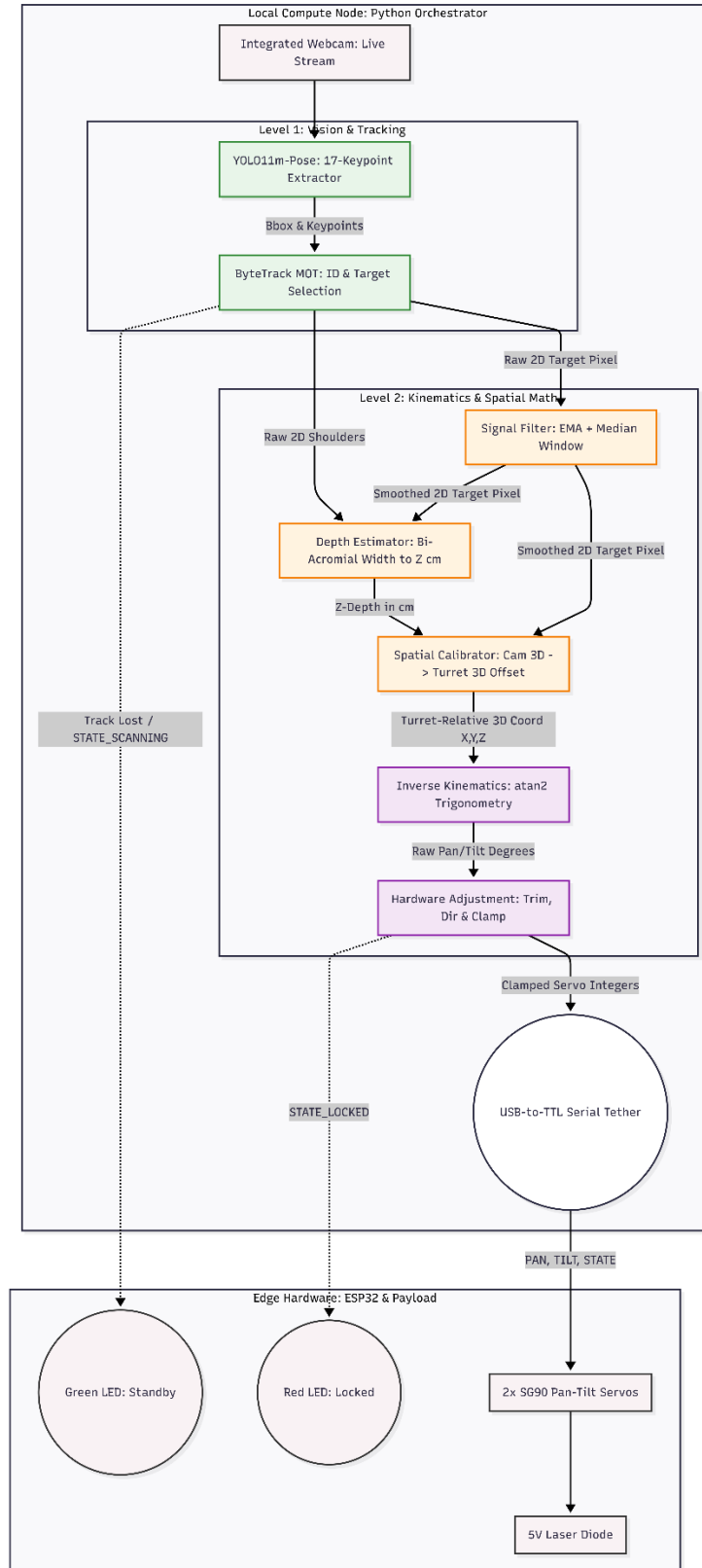


Figure 3 End-to-End Workflow of Human Detection, Tracking, and Turret Control

5. Experiments and Results

The experimentation phase was divided into three main benchmarks: pose estimation on COCO, multi-object tracking, and live inference performance. Each benchmark tested a different part of the final system. The COCO benchmark tested how accurate the pose models were, the tracking benchmark tested how stable the target identity remained across frames, and the live inference benchmark tested whether the selected model could run fast enough under real camera conditions.

For the pose estimation benchmark, the goal was to choose the model that could produce the most reliable body keypoints. This was important because the turret does not aim at a generic bounding box; it aims at a specific body point, such as the head or chest. If the keypoints are inaccurate, the calculated target point becomes wrong, which then affects depth estimation, inverse kinematics, servo movement, and laser targeting. For this reason, the benchmark used metrics such as mAP, AP50, AP75, AR, recall, precision, F1-score, mean OKS, PCK, and latency. mAP measures the overall quality of keypoint detection across different thresholds. AP50 and AP75 measure accuracy at easier and stricter matching thresholds. Precision measures how many predicted detections are correct, while recall measures how many true people or keypoints were found. F1-score combines precision and recall into one balanced score. OKS measures how close the predicted keypoints are to the ground truth while considering the size of the person, and PCK measures the percentage of keypoints that fall close enough to the correct location. Latency was also included because a very accurate model would not be useful if it was too slow for real-time control. AR measures the model's average recall across multiple thresholds, showing how consistently it can recover the true human keypoints instead of missing them.

A	B	C	D	E	F	G	H	I	J	K	L
model	mAP	AP50	AP75	AR	precision	recall	f1	mean_oks	pck	latency_ms_mean	
yolov8n	0.466567	0.726498	0.505904	0.537815	0.496292	0.558407	0.52552	0.691776	0.991446	33.76166	
yolov8s	0.566013	0.808759	0.624636	0.637925	0.541044	0.672387	0.599607	0.721848	0.994494	8.491695	
yolov8m	0.616375	0.836854	0.686274	0.690979	0.572504	0.730321	0.641854	0.737478	0.994779	13.60017	
yolo11n	0.457396	0.729696	0.492883	0.52977	0.526363	0.550063	0.537952	0.687459	0.991418	13.83073	
yolo11s	0.550697	0.800038	0.609184	0.623552	0.569132	0.657746	0.610239	0.712779	0.993992	17.00867	
yolo11m	0.615229	0.840674	0.684228	0.690397	0.615425	0.738665	0.671437	0.73478	0.994651	12.66147	
mediapipe	0.13683	0.207167	0.145659	0.16568	0.501756	0.179943	0.26489	0.702583	0.990269	41.48381	

Figure 4 Pose Estimation Benchmark Results

The COCO results showed that the medium YOLO models performed best overall. YOLOv8m achieved the highest mAP at 0.616, while YOLO11m was almost equal with 0.615. However, YOLO11m achieved the best F1-score at 0.671, higher than YOLOv8m at 0.642. YOLO11m also achieved strong AP50 at 0.841, recall at 0.739, mean OKS at 0.735, and PCK at 0.995. MediaPipe performed much worse, with only 0.137 mAP and 0.265 F1-score, showing that it was not reliable

enough for complex multi-person pose estimation. From these results, we can deduce that YOLO11m was the best choice for the project because it gave almost the highest spatial accuracy while also providing the best balance between precision and recall. The conclusion of this benchmark was that YOLO11m-Pose should be used as the final pose estimation model.

For the multi-object tracking benchmark, the goal was to choose the tracker that could maintain the same target identity across consecutive frames. This benchmark was performed on our own custom tracking dataset, created from recorded project footage and labeled with bounding boxes in YOLO format. This matters because the turret must follow one consistent person instead of jumping between detections. If the tracker loses identity or creates false positives, the servo motors may suddenly move toward the wrong target. The benchmark used MOTA, MOTP, IDF1, ID switches, mostly tracked targets, mostly lost targets, fragmentation, precision, and recall. MOTA measures the overall tracking accuracy by combining false positives, missed detections, and identity switches. MOTP measures how accurately the predicted bounding boxes align with the ground truth. IDF1 measures how consistently the tracker preserves the same identity over time. ID switches count how often the tracker changes the identity of the same target. Fragmentation measures how often a continuous track is interrupted. Precision measures how many predicted boxes are correct, while recall measures how many true target boxes are detected.

	A	B	C	D	E	F	G	H	I	J
1	tracker	MOTA	MOTP	IDF1	IDSW	MT	ML	Frag	precision	recall
2	bytetrack	0.940756	0.0998	0.635992	2	1	0	2	0.972364	0.970378
3	deepsort	0.905005	0.106094	0.615766	2	1	0	1	0.927746	0.983657

Figure 5 Multi-object Tracking Benchmark Results

The tracking results showed that ByteTrack performed better overall than DeepSORT. ByteTrack achieved a MOTA of 0.941, IDF1 of 0.636, precision of 0.972, and recall of 0.970. DeepSORT achieved a lower MOTA of 0.905, IDF1 of 0.616, precision of 0.928, and recall of 0.984.

DeepSORT had slightly higher recall, meaning it detected slightly more target boxes, but ByteTrack had much higher precision and better overall tracking accuracy. For this project, precision was especially important because false detections can cause sudden unwanted servo movement. From these results, we can deduce that ByteTrack is more suitable for a physical turret because it avoids more false positives while still keeping strong recall. The conclusion of this benchmark was that ByteTrack should be used as the final tracker.

For the live inference performance benchmark, the goal was to test whether the vision model could run fast and reliably under the actual camera conditions of the project. A model can perform well on COCO but still fail in real deployment if it becomes slow, unstable, or low-confidence on noisy webcam frames. This benchmark used the custom 1,055-frame webcam

dataset and measured FPS, latency, preprocessing time, inference time, postprocessing time, confidence mean, confidence range, and memory usage. FPS measures how many frames the system can process per second. Latency measures the delay introduced by the model for each frame. Preprocessing, inference, and postprocessing times show where the processing time is spent. Confidence mean measures how certain the model is on average, while the confidence range shows how much the model drops during difficult frames. Memory usage was measured to check whether the model was practical on the target hardware.

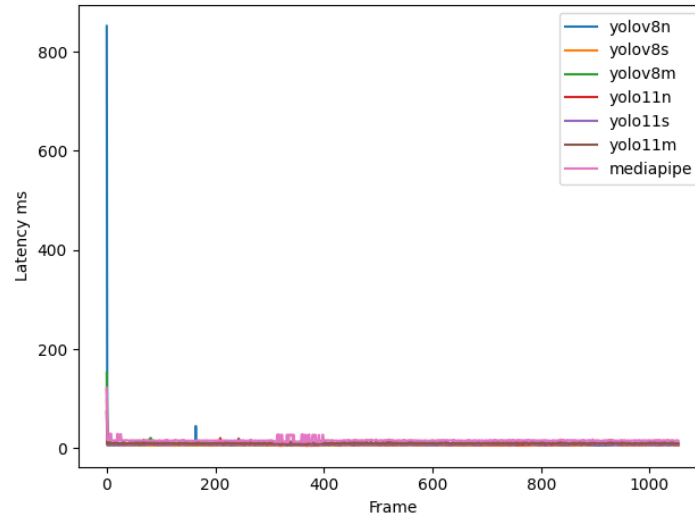


Figure 6 Latency Comparison of YOLO Pose Models and MediaPipe Across Video Frames

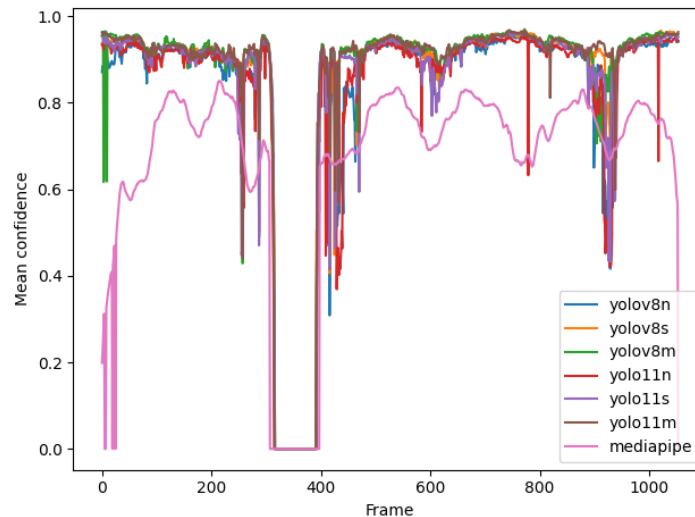


Figure 7 Mean Confidence Comparison of YOLO Pose Models and MediaPipe Across Video Frames

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
1	frame_idx	timestamp	model_nar	total_laten	preprocess	inference	postprocess	fps	num_peop	mean_con	max_confli	frame_brig	gpu_vram	cpu_ram	used_mb
2	0	0	yolov8n	851.742	9.7365	32.7351	6.2842	1.174064	1	0.87079	0.87079	108.5435	12.62793	3625.301	
3	1	0.0333	yolov8n	7.715	0.9919	5.6936	0.8581	129.6176	1	0.884614	0.884614	108.1651	12.62793	3626.016	
4	2	0.0667	yolov8n	6.067	0.6113	4.5458	0.7665	164.8261	1	0.879073	0.879073	107.5666	12.62793	3626.031	
5	3	0.1	yolov8n	6.4052	0.6187	4.7348	0.8998	156.1231	1	0.886243	0.886243	106.9513	12.62793	3626.039	
6	4	0.1333	yolov8n	6.3032	0.6099	4.779	0.7608	158.6496	1	0.898185	0.898185	106.3107	12.62793	3626.043	
7	5	0.1667	yolov8n	7.1082	0.6459	5.4754	0.8479	140.6826	1	0.905929	0.905929	105.7008	12.62793	3626.063	
8	6	0.2	yolov8n	5.8945	0.564	4.4529	0.7429	169.6497	1	0.909931	0.909931	104.9918	12.62793	3626.082	
9	7	0.2333	yolov8n	6.4781	0.5878	4.8382	0.8644	154.3662	1	0.90383	0.90383	104.8606	12.62793	3626.109	
10	8	0.2667	yolov8n	6.4636	0.5541	5.0549	0.7247	154.7125	1	0.896846	0.896846	104.9091	12.62793	3626.125	
11	9	0.3	yolov8n	8.4184	0.5808	6.3258	1.3528	118.7874	1	0.908806	0.908806	104.8188	12.62793	3626.371	
12	10	0.3333	yolov8n	7.6023	0.6332	5.9931	0.8297	131.5391	1	0.899842	0.899842	104.7893	12.62793	3626.898	
13	11	0.3667	yolov8n	7.3458	0.884	5.4716	0.8507	136.1322	1	0.889388	0.889388	104.9235	12.62793	3626.898	
14	12	0.4	yolov8n	5.9331	0.6301	4.3919	0.7538	168.546	1	0.885859	0.885859	105.0485	12.62793	3626.902	
15	13	0.4333	yolov8n	6.8929	0.5792	5.063	1.1157	145.0768	1	0.879967	0.879967	105.1411	12.62793	3626.906	
16	14	0.4667	yolov8n	6.2542	0.6232	4.6498	0.836	159.8926	1	0.882219	0.882219	105.2175	12.62793	3626.91	
17	15	0.5	yolov8n	6.4247	0.5667	4.6022	1.1129	155.6493	1	0.884819	0.884819	105.3007	12.62793	3626.914	
18	16	0.5333	yolov8n	6.1166	0.6058	4.4682	0.7447	163.4895	1	0.892581	0.892581	105.4033	12.62793	3626.926	
19	17	0.5667	yolov8n	8.4372	0.7472	6.574	0.9678	118.5227	1	0.892797	0.892797	105.5445	12.62793	3626.941	
20	18	0.6	yolov8n	8.1092	0.7244	6.4028	0.8226	123.3167	1	0.896208	0.896208	105.8095	12.62793	3626.965	
21	19	0.6333	yolov8n	6.8192	0.619	5.1343	0.9126	146.6448	1	0.893012	0.893012	106.2407	12.62793	3626.988	
22	20	0.6667	yolov8n	8.2155	0.6538	5.7113	1.7072	121.7211	1	0.888833	0.888833	106.5185	12.62793	3627.098	
23	21	0.7	yolov8n	6.2931	0.6921	4.4637	0.9866	158.9042	1	0.890282	0.890282	106.8855	12.62793	3627.117	
24	22	0.7333	yolov8n	7.9063	0.6692	5.9056	1.1803	126.4814	1	0.888534	0.888534	107.2076	12.62793	3627.117	
25	23	0.7667	yolov8n	6.747	0.7322	5.0229	0.8343	148.214	1	0.892793	0.892793	107.6333	12.62793	3627.117	
26	24	0.8	yolov8n	8.5763	0.7124	6.9054	0.8122	116.6004	1	0.908742	0.908742	108.0446	12.62793	3627.117	
27	25	0.8333	yolov8n	6.8311	0.5822	5.0745	1.0264	146.3893	1	0.924796	0.924796	108.5075	12.62793	3627.117	
28	26	0.8667	yolov8n	7.8484	0.5897	6.2373	0.8837	127.4145	1	0.934679	0.934679	109.6548	12.62793	3627.117	
29	27	0.9	yolov8n	6.94	0.6039	5.4395	0.7533	144.0922	1	0.929351	0.929351	110.8925	12.62793	3627.117	

Figure 8 Sample Benchmark Dataset Showing Frame-Level Model Performance Metrics

The live inference results showed that all YOLO models were fast enough for real-time use. YOLOv8n was the fastest, reaching about 150.66 FPS with 7.53 ms latency, while YOLOv8s reached 149.29 FPS with 6.95 ms latency. YOLO11n and YOLO11s reached 125.16 FPS and 121.65 FPS respectively. The medium models were slower but still far above the required 30 FPS threshold: YOLOv8m reached 94.75 FPS, and YOLO11m reached 96.12 FPS with about 10.80 ms latency. MediaPipe reached 62.73 FPS on CPU, but its average confidence was much lower at 0.664. YOLO11m achieved the highest average confidence at 0.858, which means it was the most stable and reliable model on the custom webcam frames. From the latency and confidence graphs, we can see that most YOLO models stayed low-latency after the initial startup frames, while confidence temporarily dropped during difficult moments such as occlusion or motion blur. The conclusion of this benchmark was that YOLO11m provided the best trade-off: it was not the fastest model, but it was still much faster than required and gave the strongest confidence stability for the final turret system.

Overall, the three benchmarks supported the same final design decision. YOLO11m-Pose was selected because it had near-best COCO accuracy, the best F1-score, the highest average confidence on the live webcam dataset, and real-time speed above the project requirement. ByteTrack was selected because it gave better tracking accuracy and fewer false positives than DeepSORT. Together, these results justified the final pipeline used in the sentry turret.

6. Discussion

The final system successfully combined computer vision, spatial calibration, inverse kinematics, embedded control, and a frontend dashboard into one working real-time pipeline. One of the strongest aspects of the project was the separation between the Python backend and the ESP32 firmware. The computer handled the expensive AI and mathematical calculations, while the ESP32 focused only on fast servo and laser control. This made the system easier to debug and kept the embedded firmware lightweight.

The vision pipeline also worked well overall. YOLO11m-Pose provided stable body keypoints and enough speed for real-time use, while ByteTrack helped maintain target identity across consecutive frames. This reduced sudden target switching and made the system more reliable during movement or short occlusions. The use of filtering also helped smooth the raw detections before converting them into servo commands.

One useful addition was the virtual laser overlay on the frontend. The final dashboard displays the green virtual laser dot directly on the live camera feed. This made it easier to understand what the system was tracking and whether the computed aiming point matched the real scene.

However, the system still has some limitations. The largest limitation is depth estimation. Since the project uses a normal 2D camera, depth is estimated from shoulder width instead of being measured directly. This works reasonably well when the shoulders are visible, but it can become less accurate when the person is rotated, partially occluded, or too close to the camera.

The original goal was to build an eye-in-hand system, where the camera would move together with the turret and observe the scene from the same moving frame as the laser. If the ESP32-CAM video stream had been stable enough, this would have reduced the need for camera-to-turret calibration because the camera and laser would share the same moving reference frame. However, due to Wi-Fi instability and latency issues with the ESP32-CAM video stream, the final implementation used an eye-to-hand setup instead. In this version, the camera remained connected to the compute node, while the turret acted as a separate moving actuator.

Laser strength also affected the calibration process. The laser was not bright or consistent enough for reliable automatic detection in the camera feed, especially under normal lighting conditions. Because of this, full automatic laser calibration was not practical, and calibration still required manual adjustment and visual verification.

Mechanical precision was another challenge. Small errors in camera alignment, servo mounting, laser placement, and bracket movement could affect the final laser position. Even if the vision model predicted the correct target point, the physical laser could still be slightly shifted because of mechanical offsets or servo backlash. This is why servo trim and calibration were important.

The system was also sensitive to lighting, motion blur, and occlusion. Although YOLO11m-Pose and ByteTrack performed well in the benchmarks, difficult scenes could still reduce confidence or cause temporary tracking loss. Fast movement could also create a delay between the detected target position and the physical servo response.

Overall, the project demonstrated that a real-time AI-assisted laser tracking turret is feasible using a local GPU, a standard camera, and an ESP32-based pan-tilt mechanism.

7. Conclusion

This project presented the design and implementation of a computer vision system for real-time human detection and tracking. The main goal was to detect a human target from camera input, estimate the target's position in the frame, and use this information as part of a larger automated sentry-style system. This directly addresses the project requirement of solving a clear and measurable computer vision problem using a real dataset, a justified approach, and quantitative evaluation.

The system relied on a deep learning-based detection approach because modern object detection models provide strong performance for real-time applications. Compared to older classical methods, deep learning models are better suited for handling variations in pose, background, lighting, and distance. Through the detection pipeline, the system was able to identify humans in input frames and produce visual outputs such as bounding boxes or body-point indicators, which helped demonstrate how the model interprets the scene.

The experimental results allowed us to evaluate the system using appropriate metrics such as precision, recall, F1-score, and mean average precision. These metrics helped measure not only whether the model detected humans correctly, but also whether it missed targets or produced false detections. In addition, visual examples of successful predictions and failure cases gave a clearer understanding of the system's strengths and limitations.

One important lesson from this project is that real-time computer vision systems depend on both model accuracy and practical implementation quality. A model may perform well in ideal images, but real-world performance can still be affected by lighting, camera quality, background complexity, hardware delay, and communication between the software and

embedded components. Because of this, the system must be evaluated not only as a detection model, but also as a complete pipeline.

Overall, the project successfully demonstrates how computer vision can be used to build an automated human detection and tracking system. While the current version provides a strong foundation, future improvements could include better depth sensing, more precise mechanical alignment, closed-loop laser feedback, and a stronger calibration workflow. The system could also be tested on a larger and more diverse dataset, optimized for embedded hardware, and further improved for faster detection and more advanced tracking across video frames. These improvements would make the turret more accurate, reliable, and suitable for real-world deployment.

8. References

Roboflow:

- <https://roboflow.com/>

Esp32 documentation/tutorials:

- <https://www.youtube.com/watch?v=RCtVxZnjPmY&t=580s&pp=ygUcZXNwMzlgY2FtIHNIldHVwIGFuZCB0dXRvcmlhbA%3D%3D>



- <https://www.youtube.com/watch?v=hSr557hppwY&pp=ygUcZXNwMzlgY2FtIHNIldHVwIGFuZCB0dXRvcmlhbA%3D%3D>



- <https://www.youtube.com/watch?v=Sb08leLWOgA&pp=ygUcZXNwMzlgY2FtIHNIldHVwIGFuZCB0dXRvcmlhbA%3D%3D>



- <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/index.html>

YOLO documentation:

- <https://docs.ultralytics.com/usage/python>

COCO dataset:

- <https://cocodataset.org/>

OpenCV documentation:

- https://docs.opencv.org/3.4/d0/de3/tutorial_py_intro.html

Mediapipe documentation:

- <https://mediapipe.readthedocs.io/>

FastAPI documentation:

- <https://fastapi.tiangolo.com/>

React documentation:

- <https://react.dev/learn>